

Contents

1	PHP Arrays	1
1.1	Indexed Arrays	1
1.2	Multidimensional arrays	3
1.3	Associative arrays	4
1.4	Here's a list of functions for working with arrays in PHP grouped by	5
1.4.1	Array creation and initialization:*	5
1.4.2	Array manipulation and modification:*	6
1.4.3	Array iteration and searching:*	6
1.4.4	Array sorting and filtering:*	6
1.4.5	Array merging and combining:*	6
1.4.6	Array accessing and manipulating elements:*	7

1 PHP Arrays

Arrays are one of the fundamental data structures in PHP, used to store collections of values. They are versatile and can be used to store various data types, including numbers, strings, objects, and even other arrays.

1.1 Indexed Arrays

Creating Arrays

There are two primary ways to create arrays in PHP:

1. **Using the square bracket syntax:** This method involves enclosing the elements within square brackets, separated by commas.

```
<?=  
$numbers = [1, 2, 3, 4, 5];  
$colors = ["red", "green", "blue"];  
$mixed = [1, "hello", true, [1, 2, 3]];
```

1. **Using the array() function:** This method explicitly creates an array and assigns it to a variable.

```
<?=  
$names = array("Alice", "Bob", "Charlie");
```

Accessing Array Elements

To access an element in an array, you can use its index, which is the position of the element within the array. Indexes start from 0, and you can access elements using the square bracket syntax.

```
<?=  
$firstNumber = $numbers[0]; // Output: 1  
$secondColor = $colors[1]; // Output: green  
$mixedElement = $mixed[3][2]; // Output: 3
```

Modifying Array Elements

You can modify the value of an existing element in an array by assigning a new value to its index.

```
<?=  
$names[0] = "John"; // Change the first name to John  
$colors[2] = "purple"; // Change the third color to purple
```

Adding Elements to Arrays

To add new elements to an array, you can use the following methods:

1. **Using the push() method:** This method appends the new element to the end of the array.

```
<?=  
$names[] = "David"; // Add David to the end of the names array  
$numbers[] = 6; // Add 6 to the end of the numbers array
```

2. **Using the array_push() function:** This method works similarly to the push() method but takes an array as the first argument and the elements to be added as the second argument.

```
<?=  
array_push($colors, "yellow", "orange");
```

3. **Using the array_merge() function:** This method merges two or more arrays into one.

```
<?=  
$newColors = array_merge($colors, ["pink", "cyan"]);
```

Removing Elements from Arrays

To remove elements from an array, you can use the following methods:

1. **Using the unset() function:** This method removes the element at the specified index from the array.

```
<?=  
unset($numbers[4]); // Remove the fourth element from the numbers array
```

2. **Using the pop() method:** This method removes the last element from the array and returns its value.

```
<?=  
$lastColor = array_pop($colors); // Remove the last color and store it in $lastColor
```

3. **Using the array_splice() function:** This method removes elements from the array and can also replace them with new elements.

```
<?=  
array_splice($colors, 1, 1, ["teal", "magenta"]); // Remove the second element and replace it with teal and magenta
```

Other Array Operations

1. **count().** The count() function is a built-in function in PHP that returns the number of elements in an array. It is a useful function for determining the size of an array. Here is an example of how to use the count() function:

```
<?=  
$numbers = [1, 2, 3, 4, 5];  
$numberOfElements = count($numbers);
```

```
echo "Number of elements: $numberOfElements";
```

2. **array_shift()** The `array_shift()` function removes the first element from an array and returns it. It is a convenient way to remove and access the first element of an array. Here is an example of how to use the `array_shift()` function:

```
<?=  
$numbers = [1, 2, 3, 4, 5];  
$firstNumber = array_shift($numbers);  
echo "First number: $firstNumber";
```

3. **array_unshift()** The `array_unshift()` function adds a new element to the beginning of an array. It is a simple way to insert an element at the beginning of an array. Here is an example of how to use the `array_unshift()` function:

```
<?=  
$numbers = [1, 2, 3, 4, 5];  
array_unshift($numbers, 0);  
echo "New array: $numbers";
```

4. **array_slice()** The `array_slice()` function extracts a specified portion of an array and returns it as a new array. It is a powerful tool for retrieving subsets of an array. Here is an example of how to use the `array_slice()` function:

```
<?=  
$numbers = [1, 2, 3, 4, 5];  
$slicedNumbers = array_slice($numbers, 2, 2);  
echo "Sliced array: $slicedNumbers";
```

These functions are essential for working with arrays in PHP. They allow you to access, modify, and manipulate arrays effectively.

Arrays are versatile and essential data structures in PHP, allowing you to store, organize, and manipulate collections of data efficiently.

1.2 Multidimensional arrays

Multidimensional arrays in PHP are arrays that contain other arrays as elements. This allows you to store and organize data in a hierarchical structure, making it well-suited for representing data with nested relationships.

Creating Multidimensional Arrays

Multidimensional arrays can be created using the square bracket syntax, nesting one or more arrays within another. For example, the following code creates a 2D array:

```
<?=  
$students = [  
    ["Alice", "12345", "English", "History", "Math"],  
    ["Bob", "67890", "Science", "Art", "Computer Science"]  
];
```

In this example, the `students` array contains two elements, each representing a student's information. Each student's information is stored in another array within the `students` array.

Accessing Elements in Multidimensional Arrays

To access an element in a multidimensional array, you can use nested indexing, where each index corresponds to the position of the element in the corresponding subarray. For example, the following code accesses the first student's name from the `students` array:

```
<?=  
$firstName = $students[0][0];  
echo "First name: $firstName";
```

Modifying Elements in Multidimensional Arrays

You can modify an element in a multidimensional array by assigning a new value to its index. For example, the following code changes the second student's math grade to 100:

```
<?=  
$students[1][4] = 100;  
echo "Second student's math grade: " . $students[1][4];
```

Adding Elements to Multidimensional Arrays

To add new elements to a multidimensional array, you can use the same methods as with one-dimensional arrays. For instance, the following code adds a new student to the `students` array:

```
<?=  
$students[] = ["Charlie", "98765", "Spanish", "Economics", "Music"];
```

Removing Elements from Multidimensional Arrays

To remove elements from a multidimensional array, you can use the `unset()` function, followed by the nested indices of the element to be removed. For example, the following code removes the first student's information from the `students` array:

```
<?=  
unset($students[0]);
```

Multidimensional arrays offer a powerful way to organize and store complex data structures in PHP. They are particularly useful for representing nested relationships, such as employee hierarchies, product categories, or network topologies.

1.3 Associative arrays

Associative arrays in PHP are arrays whose elements are associated with keys instead of numeric indices. This allows you to store and retrieve data based on meaningful labels rather than numerical positions.

Creating Associative Arrays

Associative arrays can be created using the same square bracket syntax as regular arrays, but instead of using numeric indices, you use key-value pairs enclosed in curly braces. For example, the following code creates an associative array that stores student information:

```
<?=  
$student = [  
    "name" => "Alice",  
    "age" => 20,  
    "course" => "Computer Science"  
];
```

In this example, the `student` array has three elements, each represented by a key-value pair. The key is the label for the data, and the value is the actual data itself.

Accessing Elements in Associative Arrays

To access an element in an associative array, you use the key instead of the index. For example, the following code accesses the student's name from the `student` array:

```
<?=  
$firstName = $student["name"];  
echo "Student's name: $firstName";
```

Adding Elements to Associative Arrays

To add new elements to an associative array, you simply add new key-value pairs. For instance, the following code adds the student's GPA to the `student` array:

```
<?=  
$student["gpa"] = 3.9;  
echo "Student's GPA: " . $student["gpa"];
```

Removing Elements from Associative Arrays

To remove an element from an associative array, you use the `unset()` function and provide the key of the element to be removed. For example, the following code removes the student's age from the `student` array:

```
<?=  
unset($student["age"]);  
echo "Student's age: " . $student["age"];
```

Associative arrays are a versatile and efficient data structure for storing and retrieving information based on meaningful labels. They are commonly used in various applications, such as user profiles, product catalogs, and configuration settings.

1.4 Here's a list of functions for working with arrays in PHP grouped by

functionality:

1.4.1 Array creation and initialization:*

- `array()`: Creates an empty array or initializes an array with a set of values.
- `count()`: Returns the number of elements in an array.
- `in_array()`: Checks whether a specific value exists in an array.

1.4.2 Array manipulation and modification:*

- `array_push()`: Adds one or more elements to the end of an array.
- `array_pop()`: Removes and returns the last element of an array.
- `array_shift()`: Removes and returns the first element of an array.
- `array_unshift()`: Adds one or more elements to the beginning of an array.
- `array_splice()`: Removes and replaces elements from an array.
- `isset()`: Checks whether a specific key exists in an array.
- `empty()`: Checks whether an array is empty.
- `unset()`: Removes one or more elements from an array.

1.4.3 Array iteration and searching:*

- `foreach()`: Iterates over the elements of an array.
- `array_keys()`: Returns an array of all the keys in an array.
- `array_values()`: Returns an array of all the values in an array.
- `array_search()`: Finds the first occurrence of a specific value in an array.

1.4.4 Array sorting and filtering:*

- `sort()`: Sorts an array in ascending order.
- `rsort()`: Sorts an array in descending order.
- `asort()`: Sorts an associative array by value.
- `arsort()`: Sorts an associative array by value in descending order.
- `krsort()`: Sorts an associative array by key.
- `ksort()`: Sorts an associative array by key in descending order.
- `array_filter()`: Filters an array based on a callback function.

1.4.5 Array merging and combining:*

- `array_merge()`: Merges two or more arrays into one.
- `array_slice()`: Extracts a portion of an array to create a new array.
- `array_unique()`: Removes duplicate values from an array.
- `array_fill()`: Creates a new array with a specified value and a range of keys.

1.4.6 Array accessing and manipulating elements:*

- `array_keys()`: Returns an array of all the keys in an array.
- `array_values()`: Returns an array of all the values in an array.
- `array_chunk()`: Divides an array into smaller subarrays.
- `array_combine()`: Creates an associative array from two arrays of keys and values.
- `array_map()`: Applies a callback function to each element in an array.
- `array_reduce()`: Reduces an array to a single value using a callback function.

These functions provide a comprehensive set of tools for working with arrays in PHP, covering a wide range of tasks from creating and manipulating arrays to searching, sorting, and filtering them.